

课程笔记

CS336 第 6 讲：Kernel、Triton 与 “让 GPU 真正跑快”

Codex 基于 Stanford CS336 2025 第 6 讲视频、字幕与官方课程材料重写

2026-04-07



视频作者/频道：Stanford Online

发布日期：2025 年 05 月 08 日

视频时长：1:20:22

视频链接：<https://www.youtube.com/watch?v=E8Mju53VB00>

目录

1 这讲在系统主线中的位置	2
1.1 从 A100 / H100 规格到真实 workload	2
1.2 本章小结	2
2 先别优化，先 benchmark / profile	2
2.1 为什么 benchmarking 比“看 spec sheet”更重要	2
2.2 Benchmark 解决的是“有多快”，profiling 解决的是“快慢来自哪”	2
2.3 把这套工具链真正用起来	3
2.4 为什么矩阵乘法看起来简单，性能却并不总是可预测	3
2.5 本章小结	3
2.6 官方脚本补充：benchmarking / profiling 到底落在哪些工具上	3
3 Arithmetic Intensity：把所有优化语言统一起来	4
3.1 为什么课程要把“算术强度”放在整讲中央	4
3.2 为什么矩阵乘法通常是 compute-bound，而很多别的操作不是	4
3.3 本章小结	5
4 六种常见提速手段：本质都在和 memory 搏斗	5
4.1 低精度：先从少搬一点数据开始	5
4.2 operator fusion：不要把仓库物流当成默认代价	5
4.3 recomputation：有时重算比存下再读更划算	6
4.4 coalescing 与 tiling：真正的大头	6
4.5 本章小结	7
5 CUDA 与 Triton：为什么课程要你往下看到 kernel 代码	7
5.1 为什么手写一个简单 CUDA GeLU 仍然很有价值	7
5.2 PTX、load_inline 与 thread coarsening：为什么这讲会一路下钻到汇编痕迹	7
5.3 五路实现对比：manual / PyTorch / compiled / CUDA / Triton	7
5.4 Triton 的真正价值：把大量底层机械活自动化，但不抹掉硬件思维	8
5.5 从 load_inline 到五种写法	8
5.6 XLA 作为更上层的编译参照	8
5.7 本章小结	8
6 总结与延伸	8

1 这讲在系统主线中的位置

第 5 讲已经解释了 GPU 不是魔法，而是一种受 memory hierarchy 强烈约束的吞吐机器。第 6 讲继续往下一层走：既然我们已经知道 GPU 为什么会快、为什么会慢，那么**怎样真正写出或理解一个高性能 kernel**？这就是这讲的中心。

课程把任务拆成两半：一半是 benchmarking / profiling，另一半是 CUDA / Triton 视角下的 kernel 编写。这个拆法很合理，因为你不先知道“慢在哪里”，就没有资格讨论“该怎么改快”。

第 6 讲的中心命题

高性能 kernel 不是“把一段数学公式翻译成 GPU 代码”那么简单，而是要把计算组织成一种符合 GPU execution model 与 memory hierarchy 的形式。

1.1 从 A100 / H100 规格到真实 workload

课程在最开始先回顾硬件：SM 数量、DRAM、L2、L1 / shared memory 等。这里的重点并不是让学生背规格，而是建立一种“现实硬件是有限资源池”的直觉。即使你有 H100，shared memory 仍然很小；即使 matmul 非常快，global memory 仍然是瓶颈；即使你能开很多线程，thread blocks 仍然要按 SM 分波次调度。

1.2 本章小结

这一讲不是在重复 GPU 课，而是在为 kernel 设计建立实践入口：你不只是需要知道 GPU 有什么组件，还要知道这些组件会怎样真实限制你的程序。

2 先别优化，先 benchmark / profile

2.1 为什么 benchmarking 比“看 spec sheet”更重要

课程对 benchmarking 的态度很明确：你当然可以看论文、看 spec、看厂商材料，但对自己手头的库版本、硬件和 workload 来说，**没有任何东西能替代直接 benchmark / profile**。这是因为 kernel 性能高度依赖具体实现、输入形状、数据类型和运行环境。

这一点和第 2 讲的 resource accounting 一脉相承。第 2 讲给你的是一阶近似和数量级判断；第 6 讲要你开始接受另一个现实：很多运行时现象并不是纸面 FLOPs 直接决定的，而必须靠实测来验证。

2.2 Benchmark 解决的是“有多快”，profiling 解决的是“快慢来自哪”

课程区分得很清楚：

- **benchmarking** 给的是 end-to-end wall-clock time。
- **profiling** 则帮你看清具体哪些 op、哪些 kernel、哪些同步点在吃时间。

这个区分非常重要。只 benchmark 不 profile，你只能知道 A 比 B 快，却不知道原因；只 profile 不 benchmark，你可能知道热点在哪，却不知道这些热点对整体吞吐的实际贡献有多大。

2.3 把这套工具链真正用起来

课程把 `benchmark / profile` 讲得很具体,而不是停留在口号上。做基准测试时,`torch.utils.benchmark` 是一个现成工具,但课堂上仍然保留了一个轻量的 `benchmark` 包装器,把 `warmup`、`CUDA synchronize` 和 `trial` 平均显式写出来,这样你能看见每一步到底在做什么。做 profiling 时,`torch.profiler` 可以直接揭示哪些 `CUDA kernels` 在吃时间,课程里的 `MLP` 例子还会配合 `flame graph` 看栈信息。

如果你自己在写 `CUDA` 扩展,`CUDA_LAUNCH_BLOCKING=1` 也属于同一条工具链:它能让错误在发生的调用点暴露出来,而不是拖到后面的异步 `kernel` 才一起炸掉。课程明确提到作业里还会继续用 `Nsight` 往更底层看,因此这是一条从 `wall-clock` 到 `profiler` 再到 `tracer` 的下钻路径,而不是彼此割裂的工具列表。

2.4 为什么矩阵乘法看起来简单,性能却并不总是可预测

课程专门用 `square matmul` 和一个小 `MLP` 做 `benchmark`,随后演示不同维度、层数、`batch size` 下时间变化并不总是线性可预测。这再次说明:真实 `CUDA kernel` 会受到 `tile` 形状、调度、对齐、`occupancy` 和库内实现细节的影响。

关于 kernel 性能的一个常见误区

“数学上复杂度一样,所以速度应该差不多”在 `GPU` 上经常不成立。相同数量级的操作,如果访存模式、`tile` 对齐或 `kernel` 调度方式不同,最终时间可能差很多。

2.5 本章小结

优化之前先 `benchmark / profile`,并不是保守做法,而是唯一靠谱的起点。因为你要面对的不是抽象程序,而是某个具体环境里的某个具体 `kernel`。

2.6 官方脚本补充: benchmarking / profiling 到底落在哪些工具上

`Spring 2025` 的官方 `lecture script` 在这部分其实比现有讲义更“可执行”。它不是抽象地说“要 `benchmark`、要 `profile`”,而是明确给出了一条工具链:

- 用 `torch.utils.benchmark` 做可重复的 `wall-clock benchmark`;
- 用 `torch.profiler` 看 `kernel` 级执行时间、调用关系与热点;
- 在需要强制同步定位错误时,显式打开 `CUDA_LAUNCH_BLOCKING=1`;
- 在 `profiler` 输出上配合 `flame graph / trace viewer` 去看调用栈,而不是只看一串平铺的时间表。

这条工具链之所以值得写进教材,是因为它把“`benchmarking` 回答有多快, `profiling` 回答为什么快或慢”这句话真正落到了操作层。对系统研究者而言,这一步非常重要:没有工具链,方法论就很容易退化成口号。

3 Arithmetic Intensity: 把所有优化语言统一起来

3.1 为什么课程要把“算术强度”放在整讲中央

第 6 讲最重要的统一指标就是 arithmetic intensity:

$$I = \frac{\text{\#FLOPs}}{\text{\#Bytes}}$$

课程给出的解释非常直接：如果这个比值高，操作更可能是 compute-bound；如果这个比值低，操作更可能是 memory-bound。它之所以重要，是因为几乎所有 kernel 优化技巧，最终都可以翻译成“怎样提高或保护算术强度”。

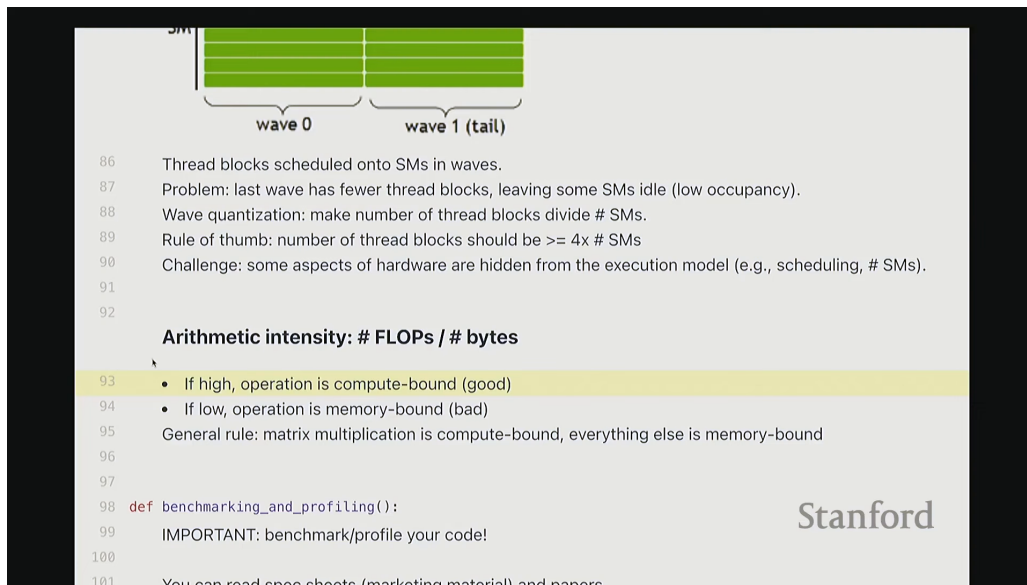


图 1: 课程直接把 arithmetic intensity 放到中间位置：高表示更接近 compute-bound，低表示更容易被 memory-bound 卡死。¹

3.2 为什么矩阵乘法通常是 compute-bound，而很多别的操作不是

矩阵乘法之所以特殊，是因为一次搬进来的数据会被大量重复用于乘加，因此更容易把计算做“厚”；而 elementwise、简单激活、归一化之类的操作，往往每搬一次数据只做很少计算，因此更容易变成 memory-bound。

这就是后面 operator fusion、tiling、shared memory 这些技巧会围着 matmul 以外的操作打转的原因：它们要么试图把本来低强度的操作变得更厚一点，要么至少别让这些操作把整体 pipeline 拖得太惨。

第 6 讲最值得背下来的判断

矩阵乘法通常是 compute-bound，很多其余深度学习常见 op 更容易 memory-bound。理解这条经验，会让后面很多 kernel 设计取舍立刻变得顺理成章。

¹视频画面时间区间：约 00:07:40–00:08:00。该图来自课程视频中的屏幕画面。

3.3 本章小结

arithmetic intensity 在这讲里起到的是“元语言”作用。它让你不用把每个技巧都孤立记忆，而是统一用“更多计算 / 更少搬运”的尺度去理解它们。

4 六种常见提速手段：本质都在和 memory 搏斗

4.1 低精度：先从少搬一点数据开始

课程再次强调低精度的重要性，但这次重心不是训练稳定性，而是 kernel 性能。位宽下降意味着一次内存传输带来的元素数增加，从而直接改善很多操作的算术强度。尤其在 tensor cores 已经高度优化低精度矩阵乘法的情况下，这种收益非常直接。

4.2 operator fusion：不要把仓库物流当成默认代价

课程用了一个非常好的类比：memory 像仓库，compute 像工厂。如果每做一个小操作都把中间结果运回仓库，再读出来做下一步，那整体性能很可能被物流拖死。fusion 的意义就是把本来会反复读写 global memory 的连续操作合进一个 kernel。

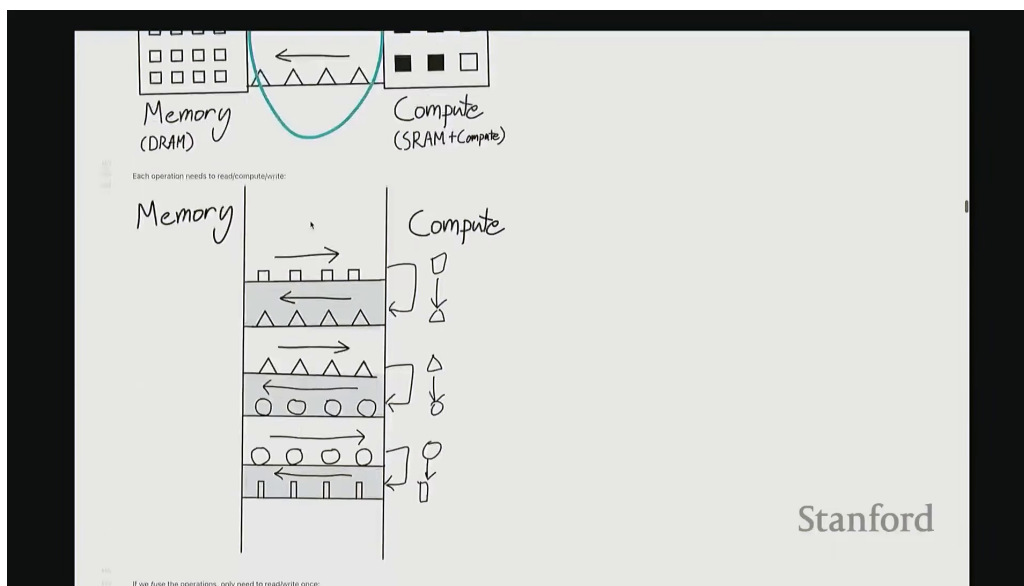


图 2: memory 是仓库，compute 是工厂。这个类比几乎可以概括第 6 讲对 kernel 优化的全部直觉。



图 3: operator fusion 的收益来自减少中间结果反复写回 / 读出 global memory, 而不是来自“数学更优雅”。²

课程用 GeLU 举例非常典型。手写版本会拆成多次 elementwise op, 每一步都可能触发独立 kernel; PyTorch 的 fused 版本则只需要更少 kernel 调用。这种差异在 GPU 上会非常大, 因为你节省的是 launch 开销和 memory round-trip。

4.3 recomputation: 有时重算比存下再读更划算

第 6 讲延续了第 5 讲对 activation checkpointing 的直觉, 但放得更通用: 有时候保留所有中间结果会导致巨大的 memory 读写开销, 倒不如在需要的时候局部重算。只要重算的 FLOPs 比搬运的代价更便宜, 这笔账就可能是合算的。

4.4 coalescing 与 tiling: 真正的大头

coalesced memory access 和 tiling 可以看作这讲最“硬核”的两部分。前者要求同一 warp 的线程尽量整齐地访问连续内存, 从而充分利用 burst mode; 后者则通过把矩阵切成 tile 并搬进 shared memory, 减少对 global memory 的重复读取。

tiling 为什么重要? 因为在非 tiled 矩阵乘法里, 很多数据会被反复从远处读进来; 一旦 tiled, 每个 tile 可以在更近的 shared memory 中被重复利用, 从而把 global memory 访问次数压低一个数量级。

为什么 tiling 是“大招”

tiling 不是一个孤立 trick, 而是把“尽量在快存储里重复利用数据”这个总原则写成程序结构的方式。很多高性能 matmul、本地 attention 和 fused kernel, 本质上都靠它在撑。

²视频画面时间区间: 约 00:42:00-00:42:20。该图来自课程视频中的屏幕画面。

4.5 本章小结

这部分最关键的 takeaway 是：低精度、fusion、recomputation、coalescing、tiling 等技巧并不分裂，它们都在围绕同一个目标工作：让计算更厚，让搬运更少，让数据尽量待在更近的存储层里。

5 CUDA 与 Triton：为什么课程要你往下看到 kernel 代码

5.1 为什么手写一个简单 CUDA GeLU 仍然很有价值

课程先让学生看一个简单的 CUDA GeLU 实现，这件事的意义不在于“未来你必须全部手写 CUDA”，而在于把抽象层扒开，让你看到 kernel 最底层到底在做什么：线程索引怎样决定处理哪个元素、grid / block 怎样映射、边界怎样处理、输出张量怎样写回。

这一步很重要，因为只有理解过这层结构，你才会真正知道 PyTorch fused op 或 Triton kernel 到底替你自动完成了哪些事情。

Listing 1: 课程里 CUDA / Triton 视角的最小 kernel 骨架：先确定索引，再做局部计算

```
pid = program_id(0)
offsets = pid * BLOCK + arange(0, BLOCK)
mask = offsets < n_elements
x = load(x_ptr + offsets, mask=mask)
y = gelu(x)
store(y_ptr + offsets, y, mask=mask)
```

这段伪代码的教学作用很明确。它把课程里反复强调的 kernel 实现骨架浓缩成四步：先用 `program_id` 或 `thread/block` 索引确定谁负责哪段数据，再按 `mask` 处理边界，再在寄存器或更近的层级完成局部计算，最后统一写回输出。也正因为这个骨架很清楚，课程后面才会继续讨论 fusion、tiling 和 Triton 编译器到底替我们省掉了哪些机械活。

5.2 PTX、load_inline 与 thread coarsening：为什么这讲会一路下钻到汇编痕迹

官方 script 在后半段补了一个很好的教学层次：不仅展示 CUDA / Triton kernel，还进一步让学生去看 PTX 级别的痕迹。这里最重要的不是记忆所有 PTX 指令，而是理解三类信息：

- 像 `ld.global`、`st.global` 这样的指令直接暴露了 global memory 读写；
- 像 `%ctaid.x`、`%tid.x` 这样的寄存器，直接告诉你 grid/block/thread 索引是如何落到实际执行单元上的；
- 通过 `load_inline`、编译后 PTX 输出和 Triton tutorial，可以把“高级 kernel 写法”与“底层访存行为”串起来看。

slides 还强调了 **thread coarsening** 这件事：有时让一个线程处理不止一个元素，会降低 occupancy，但能换来更高的数据复用或更少的控制开销。这个例子非常适合教材，因为它提醒读者：高 occupancy 不是唯一目标，真正目标是总吞吐。

5.3 五路实现对比：manual / PyTorch / compiled / CUDA / Triton

官方脚本里还有一条很值得保留的结构线：用同一个操作，依次比较手写 Python 版本、PyTorch eager 版本、compiled 版本、自定义 CUDA 版本和 Triton 版本。这样做的教学价值极高，因为它让学生看到“优化”不是一个二元开关，而是一条逐层下钻的抽象梯度。

这条五路 taxonomy 也解释了为什么 2026 最新 offering 会把第 6 讲标题升级成 *Kernels, Triton, XLA*。这里的 XLA 不一定意味着整讲都会细讲 JAX 编译器，而是说明课程已经更明确地把“图编译 / kernel lowering / 更低层 IR”纳入同一条性能工程谱系：从 eager 程序，到 compiled graph，到 custom kernel，到 PTX 痕迹，都是同一个问题的不同抽象面。

5.4 Triton 的真正价值：把大量底层机械活自动化，但不抹掉硬件思维

课程对 Triton 的定位非常准确。它不是让你“忘掉硬件”，而是让你以更高层的方式表达 kernel，同时把 memory coalescing、shared memory 管理、部分调度细节交给编译器。也就是说，**Triton 降低了进入 kernel 世界的门槛，但并没有取消对 hardware-aware 思维的要求。**

为什么 Triton 特别适合 CS336

这门课想让学生真正理解 kernel，但又不想让所有人都先被 CUDA 语法细节淹没。Triton 恰好处在一个好位置：你仍然在思考 block、tile、访存与布局，但可以用更高层的写法落地。

5.5 从 load_inline 到五种写法

课程把 GeLU 放在同一条比较链上，是为了让你看到“从最手工到最自动”的连续谱。`load_inline` 的作用，是把一段 CUDA/C++ 代码直接编译并绑定成可调用的 Python 模块，这样你就能很快把手写 kernel 接进实验流程里。它的意义不是让你永远手写扩展，而是让你看清楚 CUDA 层到底暴露了哪些控制权，以及这些控制权会怎样影响性能。

在这个例子里，课程把五种写法并排比较：manual、PyTorch、compiled、CUDA、Triton。最直接的教学结论是，manual 只是基线；PyTorch 的 builtin 版本和 `'torch.compile'` 的版本都能显著快于 manual；CUDA 版本通常比 manual 好很多，但不一定追得上库内 fused kernel；Triton 和 `'torch.compile'` 则把“写高层程序”与“得到高性能 kernel”之间的距离拉得更近。这里的重点不是死记一个固定排序，而是理解这些实现对应的是不同层级的抽象和 lowering 路径。

5.6 XLA 作为更上层的编译参照

最新 offering 的标题已经把 XLA 并列进来，这不是要把这一讲改成 JAX 课，而是在提醒你：这一层的思想并不只属于 Triton。课程末尾提到 `'torch.compile'`，也顺带提到 JAX 的 XLA compiler；它们都在做同一件事，就是把高层张量程序 lower 成更少 kernel launch、更少读写、更符合硬件偏好的执行形态。换句话说，XLA 只是更大编译栈里的一个自然参照点，而不是对前面 GPU 直觉的替代。

5.7 本章小结

第 6 讲最后从 benchmarking/profiling 一路走到 CUDA 和 Triton，其实是在完成一个非常完整的闭环：先测哪里慢，再理解慢的根源，最后把这种理解变成更好的 kernel 设计。

6 总结与延伸

第 6 讲最重要的结论可以压缩成四点。

第一，**benchmark / profile 是 kernel 优化的起点，而不是收尾**。没有实测，你只是猜。

第二, **arithmetic intensity** 是统一理解 kernel 优化的核心尺度。高强度意味着更接近 compute-bound, 低强度则意味着你更容易被 memory 拖死。

第三, **GPU 提速技巧的共同本质, 是减少或重组数据搬运**。fusion、recomputation、coalescing、tiling 全都属于这个框架。

第四, **Triton 的价值不在于隐藏硬件, 而在于让你能用更合理的抽象直接表达 hardware-aware kernel**。

我对第 6 讲的压缩提醒

如果你还把 kernel 优化理解成“把公式翻译成更底层代码”, 那你会一直抓不到重点。真正的重点是: 你有没有把数据组织成 GPU 愿意高效处理的形状。

这讲之后, 自然要进入并行训练与更复杂的系统层, 因为单个 GPU 上的 kernel 理解, 最终会扩展成多卡、多机上的通信与分片问题。也就是说, 第 6 讲是从“硬件直觉”走向“分布式系统直觉”的最后一站。